

开发指南

HTTP API 调用

 Copy page

智谱AI 提供基于 RESTful 架构的应用程序接口，通过标准的 HTTP 协议与智谱AI 的模型服务进行交互。无论您使用什么编程语言或开发框架，都可以通过 HTTP 请求来调用智谱AI 的各种 AI 模型。

核心优势



跨平台兼容

支持所有支持 HTTP 协议的编程语言和平台



标准协议

基于 RESTful 设计，遵循 HTTP 标准，易于理解和使用



灵活集成

可以集成到任何现有的应用程序和系统中



实时调用

支持同步和异步调用，满足不同场景需求

获取 API Key

1. 访问 [智谱AI 开放平台](#)
2. 注册并登录您的账户
3. 在 [API Keys](#) 管理页面创建 API Key

💡 建议将 API Key 设置为环境变量替代硬编码到代码中，以提高安全性。
>

API 基础信息

请求端点(通用API)

```
https://open.bigmodel.cn/api/paas/v4/
```

⚠️ 注意：使用 **GLM 编码套餐** 时，需要配置专属的
Coding 端点 - <https://open.bigmodel.cn/api/coding/paas/v4>
而非通用端点 - <https://open.bigmodel.cn/api/paas/v4>
注意：Coding API 端点仅限 Coding 场景，并不适用通用 API 场景，请区分使用。

请求头要求

```
Content-Type: application/json  
Authorization: Bearer YOUR_API_KEY
```

支持的鉴权方式

[API Key 鉴权](#) [JWT Token 鉴权](#)

最简单的鉴权方式，直接使用您的 API Key：

 `curl --location 'https://open.bigmodel.cn/api/paas/v4/chat/completions' \`
`--header 'Authorization: Bearer YOUR_API_KEY' \`
`--header 'Content-Type: application/json' \`
`--data '{`
 `"model": "glm-5.1",`
 `"messages": [`
 `{`
 `"role": "user",`
 `"content": "你好"`
 `}`
 `],`
`}`

基础调用示例

简单对话

```
curl --location 'https://open.bigmodel.cn/api/paas/v4/chat/completions' \
--header 'Authorization: Bearer YOUR_API_KEY' \
--header 'Content-Type: application/json' \
--data '{
  "model": "glm-5.1",
  "messages": [
    {
      "role": "user",
      "content": "请介绍一下人工智能的发展历程"
    }
  ],
  "temperature": 1.0,
  "max_tokens": 1024
}'
```

流式响应



curl -X POST -H 'Authorization: Bearer YOUR_API_KEY' -d '{"model": "glm-5.1", "messages": [{"role": "user", "content": "写一首关于春天的诗"}], "stream": true}' https://open.bigmodel.cn/api/paas/v4/chat/completions' \

--header 'Content-Type: application/json' \

--data '{

"model": "glm-5.1",

"messages": [

{

"role": "user",

"content": "写一首关于春天的诗"

}

],

"stream": true

}'

多轮对话



curl -X POST -H 'Authorization: Bearer YOUR_API_KEY' -H 'Content-Type: application/json' -d '{

--header 'Authorization: Bearer YOUR_API_KEY' \

--header 'Content-Type: application/json' \

--data '{

"model": "glm-5.1",

"messages": [

{

"role": "system",

"content": "你是一个专业的编程助手"

},

{

"role": "user",

"content": "什么是递归? "

},

{

"role": "assistant",

"content": "递归是一种编程技术，函数调用自身来解决问题..."

},

{

"role": "user",

"content": "能给我一个 Python 递归的例子吗? "

}

]

}'

常用编程语言示例

[Python](#)

[JavaScript](#)

[Java](#)

```
def call_zhipu_api(messages, model="glm-5.1"):
    url = "https://open.bigmodel.cn/api/paas/v4/chat/completions"

    headers = {
        "Authorization": "Bearer YOUR_API_KEY",
        "Content-Type": "application/json"
    }

    data = {
        "model": model,
        "messages": messages,
        "temperature": 1.0
    }

    response = requests.post(url, headers=headers, json=data)

    if response.status_code == 200:
        return response.json()
    else:
        raise Exception(f"API调用失败: {response.status_code}, {response.text}")

# 使用示例
messages = [
    {"role": "user", "content": "你好, 请介绍一下自己"}
]

result = call_zhipu_api(messages)
print(result['choices'][0]['message']['content'])
```

错误处理

常见错误码

	说明	解决方案	🔍	⋮
401	未授权	检查 API Key 是否正确		
429	请求过于频繁	降低请求频率，实施重试机制		
500	服务器内部错误	稍后重试，如持续出现请联系支持		

更多错误码和解决方案请参考 [API 错误码文档](#)

实践建议

安全性

- 妥善保管 API Key，不要在代码中硬编码
- 使用环境变量或配置文件存储敏感信息
- 定期轮换 API Key

性能优化

- 实施连接池和会话复用
- 合理设置超时时间
- 使用异步请求处理高并发场景

错误处理

- 实施指数退避重试机制
- 记录详细的错误日志
- 设置合理的超时和重试次数

监控

- 监控 API 调用频率和成功率
- 跟踪响应时间和错误率
- 设置告警机制

更多资源



API 文档



技术支持



[查看完整的 API 接口文档和参数说明](#)

[获取技术支持和帮助](#)



ⓘ 建议在生产环境中使用 HTTPS 协议，并实施适当的安全措施来保护您的 API 密钥和数据传输。

[核心参数](#)

[官方 Python SDK](#)